HIVEMIND

DOCUMENTATIE PROIECT POO(C++)

Șolcă Andrei-Rareș

1. PREZENTARE

Aplicația are ca scop simularea unei flote de agenți autonomi (Drone, Roboți, Scutere) într-un oraș cu harta reprezentată ca o matrice de dimensiuni MxN. Motorul acestei aplicații (engine) rulează o simulare pe tick-uri, generează pachete în bază, actualizează agenții în timp ce decide cine este potrivit să livreze pachetul și calculează profitul obținut ținând cont de costurile de operare și de penalizări.

2. STRATEGIA DE GENERARE A HĂRȚII

Harta este o matrice de dimensiuni MxN cu celulele: Road(.), Zid(#), Bază(B), Stație(S), Client(D). Harta este gândită ca un *Strategy Pattern*, astfel încât harta să poată fi încărcată din fișier sau generată procedural.

2.1 FileMapLoader

În modul de debug, harta poate fi citită dintr-un fișier text, această metodă fiind utilă în scopul testării rapide a pathfinding-ului.

2.2 ProceduralMapGenerator

În modul procedural, harta este generată aleator, respectând parametrii din simulation_setup.txt (dimensiune, număr de clienți, de stații, număr total de pachete). Procesul a constat în:

- Inițializarea doar cu drumuri și plasarea zidurilor pe hartă (le-am dat o proporție de 20%, deoarece 25, respectiv 30%, încărcau harta prea mult).
- am verificat pozițiile valide (să nu fie ziduri) și am plasat în mod unic baza;
- am plasat stațiile și clienții asigurându-mă că nu se vor suprapune când îi plasez pe hartă.

2.3 Validarea hărții

După generare, harta trebuie validată printr-un BFS, pornind de la bază, verificându-se că se poate ajunge de la bază la toate stațiile, respectiv toți clienții. Dacă o stație sau un client nu ar fi accesibil, harta se regenerează.

3. ALGORITMUL HIVEMIND

HiveMind decide maximizarea profitului. La fiecare tick, hotărăște ce pachet va fi alocat fiecărui agent disponibil și dacă agentul merge direct la destinație sau se duce să se încarce, cu riscul de a întârzia, dar măcar nu pierde pachetul.

În decizia alocării pachetului, folosesc o structură numită `Opportunity` cu atribute ce mă ajută la calcularea unui scor. Acel scor va depinde de rută, de distanța față de stații, de bateria pe care o are și de caracteristicile agentului. După calculare, aleg agentul cu scorul cel mai bun să livreze pachetul.

La fiecare tick, stările agenților pot evolua (ei se pot afla în 4 stări: IDLE, MOVING, CHARGING, DEAD). Având în vedere penalizările aduse de un agent mort, voi verifica bateria fiecărui agent care primește un pachet și trebuie să îl livreze, astfel încât să știe să meargă la cea mai apropiată stație sau în bază pentru a se încărca. Din acest punct nu va mai consuma (costPerTick) și își va asuma întârzierea decât să moară și să nu livreze pachetul.

Analizând ruta până la clientul care are de primit pachetul, voi alege Robotul pentru pachete aflate la distanțe mici sau cu deadline mic/mediu, deoarece nu consumă atât de mult ca o Dronă, pe care o voi alege pentru urgențe sau trasee lungi.

De asemenea, fixez numărul de agenți care se mișcă în același timp pentru a evita creșterea haotică a costurilor de operare.

4. CUM RULEZ PROIECTUL?

Aplicația va prelua date din fișierul “simulation_setup.txt”, generează/rulează harta și o va simula pentru numărul maxim de ticks scris în fișier. La final, raportul cu informații de după simulare va fi scris în fișierul “simulation.txt”.

De asemenea, am afișat și harta aferentă simulării precedente pentru a o putea evalua.

5. DIAGRAMA UML DE CLASE

